UNIT 1 – Software Engineering

Topic 1: Introduction to Software Engineering

What is Software?

Software is a collection of:

- **Programs** (instructions to the computer)
- **Data** (information used by programs)
- **Documentation** (user manuals, design docs, etc.)

👉 Software tells **hardware what to do**.

What is Engineering?

Engineering is:

The systematic application of scientific principles to design, build, and maintain solutions efficiently, economically, and reliably.

Key ideas:

- Systematic
- Disciplined
- Uses proven methods
- Focus on quality and cost

Definition of Software Engineering

Software Engineering is:

The application of engineering principles to the **development, operation, and maintenance of software**.

📌 *IEEE Definition* (important for exams):

Software engineering is the application of a **systematic, disciplined, and quantifiable approach** to the development, operation, and maintenance of software.

Why Software Engineering is Needed?

Early software was:

- Small
- Written by one person
- No formal process

Modern software is:

- Large & complex
- Multi-developer
- Time-bound
- Safety & security critical

➡️ **Software Engineering ensures:**

- Correctness
- Reliability
- Maintainability
- Scalability

Goals of Software Engineering

1. **High Quality Software**
2. **Delivered on Time**
3. **Within Budget**
4. **Meets User Requirements**
5. **Easy to Maintain & Upgrade**

Key Components of Software Engineering

Software Engineering focuses on **four P's**:

1. **People** – Developers, managers, users
2. **Product** – The software being built
3. **Process** – Steps followed to build software
4. **Project** – Planning, scheduling, tracking

Characteristics of Good Software

A well-engineered software system should be:

- **Correct** – Does what is required
- **Reliable** – Works consistently
- **Efficient** – Uses resources well
- **Usable** – Easy to use
- **Maintainable** – Easy to modify
- **Portable** – Runs on different platforms

Difference: Programming vs Software Engineering

Programming	Software Engineering
Writing code only	Complete development process
Individual focus	Team-based
No formal process	Defined models & standards
Small programs	Large-scale systems

Topic 2: Importance of Software Engineering as a Discipline

Why Software Engineering is Important?

Software today controls **banks, hospitals, airplanes, defence systems, satellites, mobile apps, and AI systems**.
A small software failure can cause **huge financial loss or even loss of life**.

➡️ Software Engineering provides a **structured, disciplined, and reliable way** to build such systems.

Key Reasons for the Importance of Software Engineering

1. Manages Software Complexity

Modern software systems are:

- Very large
- Highly complex
- Built by teams, not individuals

Software Engineering:

- Breaks the system into modules
- Uses design methods & architectures
- Controls complexity systematically

---

2. Improves Software Quality

Quality software must be:

- Correct
- Reliable
- Secure
- Maintainable

Software Engineering ensures quality by:

- Reviews & testing
- Standards & documentation
- Verification & validation

---

3. Ensures On-Time Delivery

Without a process:

- Projects get delayed
- Deadlines are missed

With Software Engineering:

- Proper planning
- Scheduling
- Monitoring & control

➡️ Helps deliver software **on time**.

---

4. Cost Control & Budget Management

Poorly engineered software leads to:

- Frequent rework
- High maintenance cost

Software Engineering:

- Detects errors early
- Reduces rework
- Lowers overall cost

📌 *Fixing a bug late costs much more than fixing it early.*

---

5. Supports Team-Based Development

Large projects involve:

- Designers
- Developers
- Testers
- Managers

Software Engineering provides:

- Defined roles
- Clear communication
- Documentation standards

---

6. Facilitates Maintenance & Upgrades

Software is not static.
It must be:

- Updated
- Enhanced
- Fixed

Software Engineering makes software:

- Easy to understand
- Easy to modify
- Easy to scale

---

7. Handles Changing Requirements

User requirements often change.

Software Engineering:

- Manages requirement changes
- Uses requirement management
- Prevents uncontrolled changes

---

8. Provides Standards & Best Practices

Software Engineering follows standards like:

- **IEEE standards**
- ISO standards
- Coding & documentation standards

➡️ Ensures uniformity and professionalism.

Real-World Example (Exam Friendly)

- Banking software needs **accuracy & security**
- Medical software needs **reliability**
- Air traffic control needs **zero failure**

Only Software Engineering practices can ensure this level of trust.

Advantages of Software Engineering (Summary Points)

- Reduces risk
- Improves quality

- Saves cost
- Enhances maintainability
- Enables large-scale development

UNIT 1 – Grouped Explanation

Category 1: Software Applications + Software Crisis

(These two are directly connected: *where software is used → why problems occurred*)

---

1. Software Applications

Definition

Software applications are programs designed to perform **specific tasks for users or organizations**.

Major Categories of Software Applications

1. System Software

Controls and manages computer hardware.

Examples:

- Operating Systems (Windows, Linux)
- Device Drivers
- Compilers

Purpose:
👉 Provide a platform for other software.

---

2. Application Software

Performs user-oriented tasks.

Examples:

- Banking software
- E-commerce apps
- Word processors

- Mobile apps

---

3. Engineering & Scientific Software

Used for:

- Scientific calculations
- Simulations
- Engineering design

Examples:

- MATLAB
- CAD/CAM software
- Weather forecasting systems

---

4. Embedded Software

Runs inside hardware devices.

Examples:

- Washing machines
- Cars
- Medical equipment
- Microwave ovens

📌 Must be **highly reliable**.

---

5. Web Applications

Run on browsers using internet technologies.

Examples:

- Online shopping websites
- Social media platforms
- Cloud-based tools

---

6. Artificial Intelligence Software

Simulates intelligent human behavior.

Examples:

- Expert systems
- Chatbots
- Recommendation systems
- Image recognition systems

---

7. Real-Time Software

Responds within strict time limits.

Examples:

- Air traffic control
- Missile guidance systems
- Medical monitoring systems

Why Application Variety Increased?

- Growth of internet
- Automation
- Mobile & cloud computing
- AI & data-driven systems

---

2. Software Crisis

What is Software Crisis?

Software Crisis refers to the **problems faced in software development** during the late 1960s and early 1970s due to rapidly increasing software complexity.

Main Problems Observed

- Projects delivered **late**

- Software exceeded **budget**
- Software was **unreliable**
- Difficult to maintain
- Failed to meet user requirements

Causes of Software Crisis

1. Increasing size & complexity of software
2. No proper development process
3. Poor documentation
4. Lack of project management
5. Ad-hoc coding practices

Effects of Software Crisis

- Project failures
- Financial losses
- Low user satisfaction
- Maintenance nightmares

📌 Example often quoted:

“Adding manpower to a late software project makes it later.”

Solution to Software Crisis

➡️ **Birth of Software Engineering**

Software Engineering introduced:

- Systematic development processes
- Life cycle models
- Requirement analysis
- Design methodologies
- Testing & maintenance strategies

Connection Between the Two (Very Important for Exams)

Software Applications	Software Crisis
Rapid growth in applications	Development couldn't keep up
Large, complex systems	Poor quality & failures
Mission-critical software	Need for discipline

👉 **Software Engineering emerged to handle complex software applications and eliminate the software crisis.**

Category 2: Software Processes + Software Characteristics + Software Life Cycle (SDLC)

(These always come together in exams)

---

1. Software Process

Definition

A **software process** is a **set of structured activities** required to develop a software system.

📌 In simple words:

How software is built step by step.

Generic Software Process Activities

Almost all software processes include:

1. **Specification** – What the system should do
2. **Design & Implementation** – How it will be built
3. **Validation** – Testing whether it works correctly
4. **Evolution (Maintenance)** – Modifying after delivery

Why Software Process is Important?

- Provides discipline

- Ensures repeatability
- Controls time & cost
- Improves quality

---

2. Software Characteristics

Characteristics of Software (Important Theory Question)

1. Software is Developed, Not Manufactured

- Hardware is manufactured
- Software is designed & coded

---

2. Software Does Not Wear Out

- Hardware fails due to wear
- Software fails due to **changes or bugs**

📌 Failure curve is different from hardware.

---

3. Software is Complex

- Large number of states
- Logical complexity
- Hard to visualize

---

4. Software is Mostly Custom-Built

- Unlike hardware parts
- Reuse is limited (though increasing)

---

5. Software is Invisible

- Cannot be physically seen
- Hard to measure progress

Why These Characteristics Matter?

They:

- Increase development difficulty
- Demand structured engineering methods
- Justify the need for SDLC models

---

3. Software Life Cycle (SDLC)

Definition

Software Life Cycle is the **sequence of phases** through which a software product passes from conception to retirement.

Typical SDLC Phases

1. Feasibility Study
2. Requirements Analysis
3. Design
4. Coding / Implementation
5. Testing
6. Deployment
7. Maintenance

📌 Each phase has defined outputs.

Purpose of SDLC

- Organizes development
- Reduces risk
- Improves quality
- Enables project control

Relationship Between Process, Characteristics & SDLC

Aspect	Role
Software Characteristics	Explain why software is hard
Software Process	Defines how to build software
SDLC	Implements the process in phases

Exam-Ready Short Answers

Q: What is a software process?

A software process is a structured set of activities required to develop, deliver, and maintain a software system.

Q: List characteristics of software.

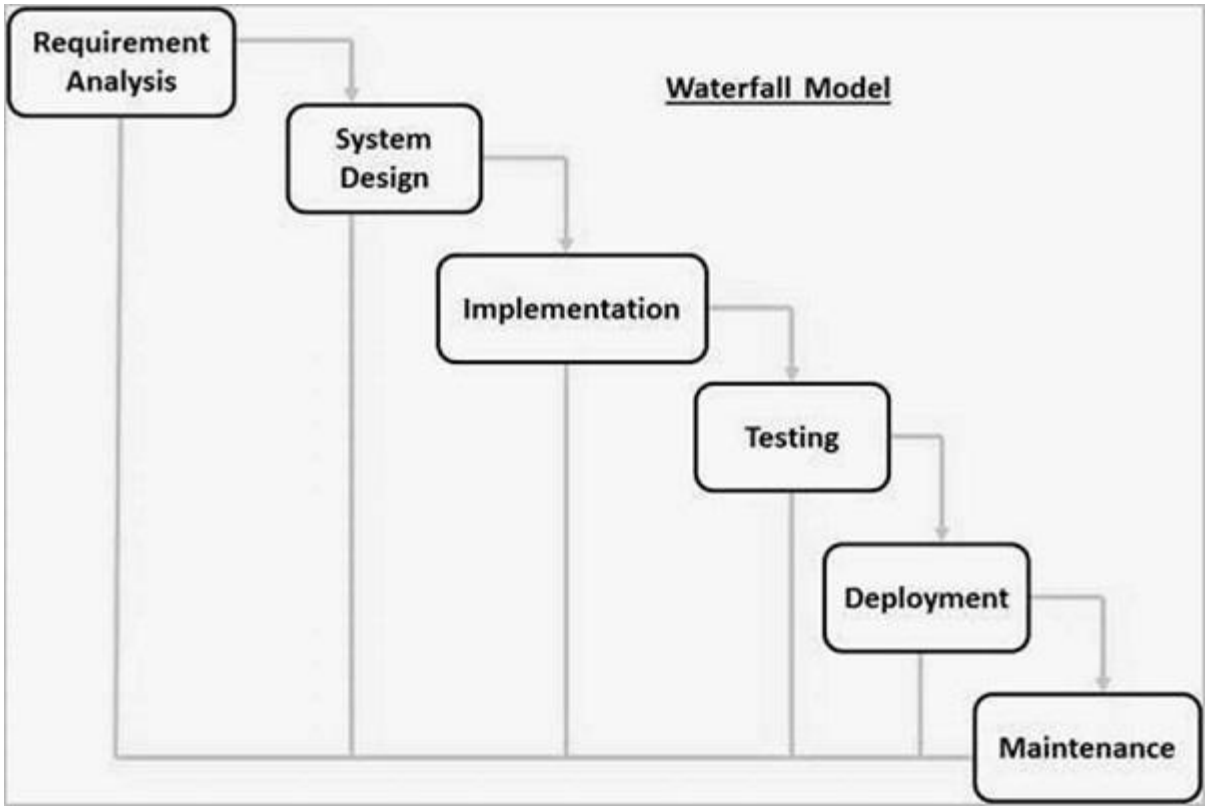
Software is developed not manufactured, does not wear out, is complex, invisible, and mostly custom-built.

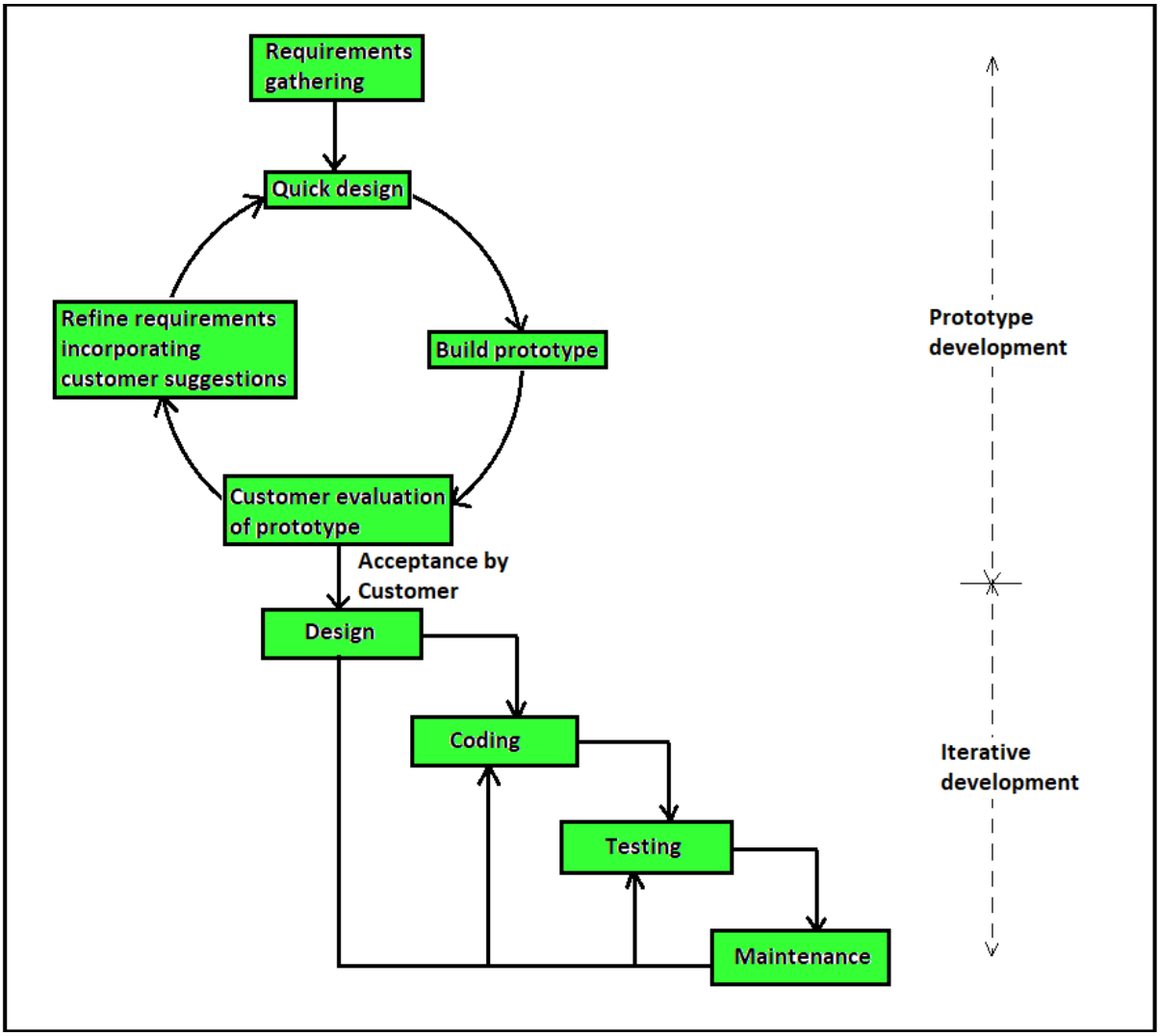
Q: What is SDLC?

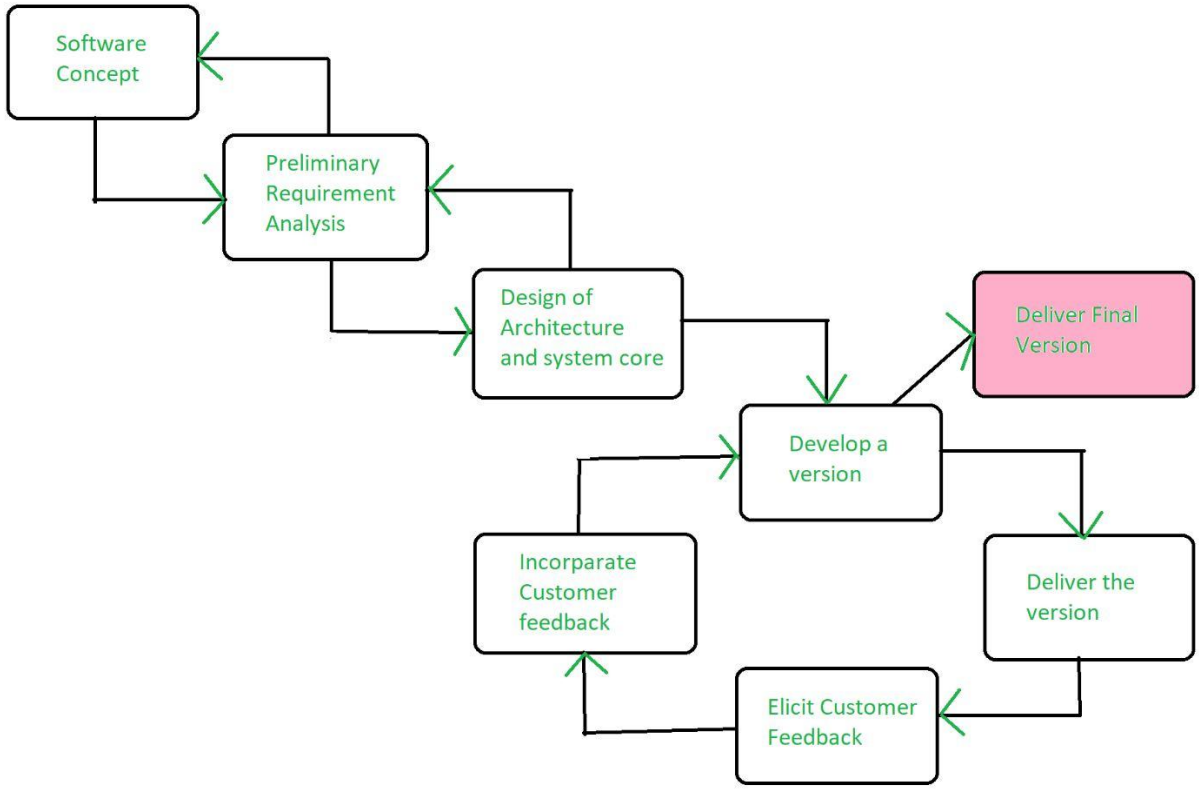
Software Development Life Cycle is a framework that describes the phases involved in developing software from requirement analysis to maintenance.

Category 3: Software Life Cycle Models

Waterfall, Prototype, Evolutionary & Spiral Model


Requirement Analysis
System Design
Implementation
Testing
Deployment
Maintenance
Waterfall Model


Requirements gathering
Quick design
Build prototype
Customer evaluation of prototype
Refine requirements incorporating customer suggestions
Acceptance by Customer
Design
Coding
Testing
Maintenance
Prototype development
Iterative development



4

4

What is a Software Life Cycle Model?

A **software life cycle model** is:

A **structured framework** that defines the **sequence of phases** and activities used to develop software.

📌 It tells **how phases are ordered and connected**.

---

1. Waterfall Model

Definition

The **Waterfall Model** is a **linear and sequential** life cycle model where each phase must be completed before the next begins.

Phases

1. Requirement Analysis
2. System Design
3. Implementation (Coding)
4. Testing
5. Deployment
6. Maintenance

➡️ Flow is **one-directional (downwards)**.

Features

- Simple & easy to understand
- Each phase has clear deliverables
- No overlapping of phases

Advantages

- Easy to manage
- Suitable for **small projects**
- Well-defined documentation

Disadvantages

- No flexibility for requirement changes
- Late testing
- High risk if requirements are wrong

When to Use

- Stable requirements
- Small, low-risk projects

---

2. Prototype Model

Definition

The **Prototype Model** involves building a **working model (prototype)** to understand user requirements clearly before actual development.

Working

1. Initial requirements gathered
2. Prototype built
3. User feedback taken
4. Prototype refined
5. Final system developed

Features

- User interaction is high
- Requirements clarified early
- Prototype may or may not be discarded

Advantages

- Better requirement understanding
- Reduces user dissatisfaction
- Early detection of errors

Disadvantages

- Poor documentation risk
- Users may think prototype is final product
- Increased cost & time

When to Use

- Requirements are **unclear**

- User interface is critical

---

3. Evolutionary Model

Definition

The **Evolutionary Model** develops software in **increments**, where each version adds new features until the complete system is built.

Key Idea

Software evolves over time.

Features

- Incremental development
- Early delivery of partial system
- Continuous user feedback

Advantages

- Flexible to changes
- Early working software
- Reduced risk

Disadvantages

- Difficult project management
- Requires skilled developers
- Architecture may degrade

When to Use

- Large systems
- Requirements change frequently

---

4. Spiral Model

Definition

The **Spiral Model** is a **risk-driven, iterative model** that combines **Waterfall + Prototyping + Iteration**.

Each Spiral Loop Includes

1. Planning
2. Risk Analysis
3. Engineering
4. Evaluation

📌 **Risk analysis is the core feature**.

Features

- Risk management focused
- Iterative & incremental
- Customer involvement at each loop

Advantages

- Best for large, complex projects
- Early risk identification
- High reliability

Disadvantages

- Expensive
- Complex to manage
- Not suitable for small projects

When to Use

- High-risk systems
- Mission-critical software

Comparison Table (Very Exam Important)

Model	Key Feature	Best Use
Waterfall	Linear & sequential	Stable requirements
Prototype	Early user feedback	Unclear requirements
Evolutionary	Incremental growth	Large, changing systems
Spiral	Risk-driven	High-risk projects

Category 4: Software Requirements Analysis & Specification + Requirement Engineering

---

1. Software Requirement

Definition

A **software requirement** is:

A condition or capability that a system must satisfy to meet user needs.

📌 In simple terms:

What the system should do.

---

2. Software Requirements Analysis & Specification

Requirements Analysis

Requirements analysis is the process of:

- Understanding user needs
- Analyzing them
- Resolving conflicts
- Defining system boundaries

Objectives of Requirements Analysis

- Identify **functional requirements**
- Identify **non-functional requirements**
- Remove ambiguity
- Ensure completeness & consistency

Types of Requirements

1. Functional Requirements

- Describe **what the system does**
- Input → Processing → Output

Examples:

- User login
- Payment processing
- Report generation

---

2. Non-Functional Requirements

- Describe **how the system performs**

Examples:

- Performance
- Security
- Reliability
- Scalability
- Usability

Requirement Specification

Requirement specification is:

The process of **documenting analyzed requirements** in a clear, structured form.

📌 Output is **SRS document** (covered later).

---

3. Requirement Engineering (RE)

Definition

Requirement Engineering is a **systematic process** of:

Eliciting, analyzing, documenting, validating, and managing software requirements.

Phases of Requirement Engineering

1. **Requirement Elicitation**
2. **Requirement Analysis**
3. **Requirement Specification**
4. **Requirement Validation**
5. **Requirement Management**

Why Requirement Engineering is Important?

- Errors here are **very costly**
- Wrong requirements → wrong software
- Most project failures occur due to poor requirements

Relationship Between Analysis & Requirement Engineering

Aspect	Role
Requirements Analysis	Understanding & refining needs
Requirement Engineering	Full lifecycle of requirements

Key Problems in Requirements

- Ambiguity
- Incompleteness

- Inconsistency
- Changing user needs

Exam-Ready Short Answers

Q: What is requirement analysis?

Requirement analysis is the process of identifying, analyzing, and documenting the needs and constraints of the software system.

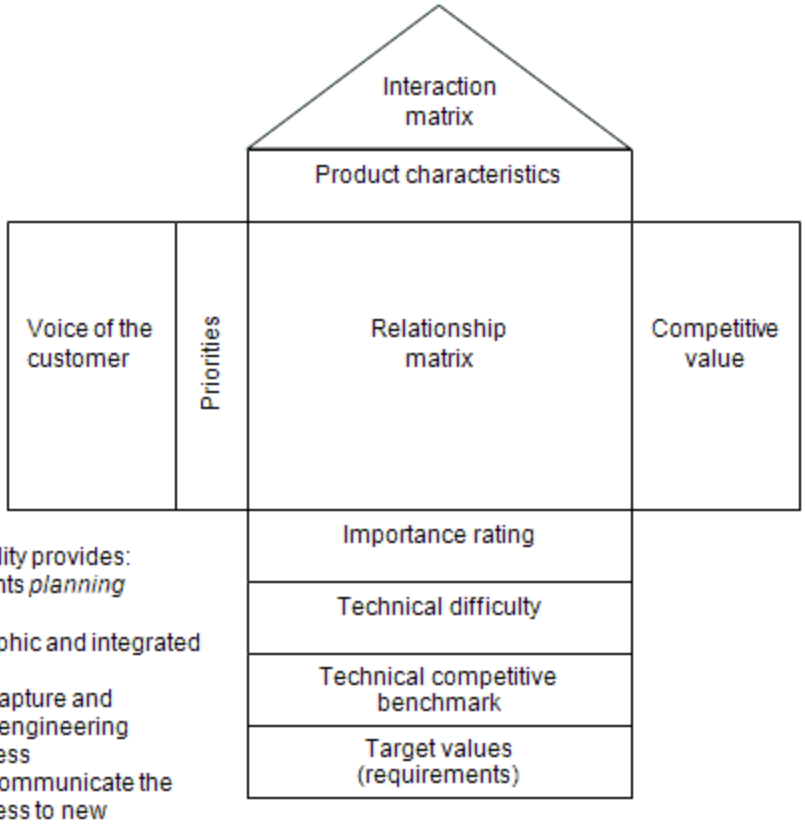
Q: Define requirement engineering.

Requirement engineering is the systematic process of gathering, analyzing, documenting, validating, and managing software requirements.

Category 5: Requirement Elicitation Techniques

FAST, QFD & Use Case Approach

Figure 1 — House of quality template and benefits



The house of quality provides:

- A requirements *planning* capability
- A tool for graphic and integrated thinking
- A means to capture and preserve the engineering thought process
- A means to communicate the thought process to new members of the QFD team
- A means to inform management regarding inconsistencies between requirements, risks, and needs of the customer

What is Requirement Elicitation?

Definition

Requirement elicitation is the process of:

Gathering requirements from stakeholders such as users, customers, managers, and domain experts.

📌 It answers:
👉 *What does the user really want?*

Why Requirement Elicitation is Important?

- Prevents misunderstanding
- Reduces rework
- Improves user satisfaction
- Forms the foundation of SRS

---

1. FAST (Facilitated Application Specification Technique)

Definition

FAST is a **team-based requirement elicitation technique** where developers and users work together in **structured workshops**.

Key Features

- Conducted in **joint sessions**
- Facilitated by a **moderator**
- Encourages open discussion
- Focus on early requirement clarification

FAST Process

1. Participants selected (users, developers, managers)
2. Problem defined
3. Requirements brainstormed
4. Conflicts resolved
5. Preliminary requirements documented

Advantages

- Faster requirement gathering
- Reduces communication gap
- Improves user involvement

Disadvantages

- Requires skilled facilitator
- Time-consuming sessions
- Not suitable for very large teams

Exam Tip

📌 FAST is also called **Joint Application Development (JAD)**.

---

2. QFD (Quality Function Deployment)

Definition

QFD is a technique used to:

Transform customer needs into technical requirements.

📌 Main tool: **House of Quality**.

Key Idea

Voice of customer → Technical features

House of Quality Components

- Customer requirements (WHATs)
- Technical descriptors (HOWs)
- Relationship matrix
- Priority ratings

Features

- Customer-focused
- Quantitative prioritization
- Improves product quality

Advantages

- Clear mapping of user needs
- Helps in prioritization
- Reduces ambiguity

Disadvantages

- Complex to construct
- Time-consuming
- Needs domain expertise

When QFD is Used

- High customer involvement
- Quality-critical systems

---

3. Use Case Approach

Definition

A **Use Case** describes:

A sequence of actions between a **user (actor)** and the system to achieve a goal.

Key Elements

- **Actor** – User or external system
- **Use Case** – Functionality
- **System Boundary**

Use Case Diagram

Graphical representation showing:

- Actors
- Use cases
- Interactions

Advantages

- Easy to understand

- User-centric
- Helps in functional requirement identification

Disadvantages

- Does not capture non-functional requirements well
- May miss internal processing logic

Where Use Cases are Best

- User-driven systems
- Interactive applications

Comparison Table (Very Important for Exams)

Technique	Focus	Best For
FAST	Group discussion	Early requirement clarity
QFD	Customer → Technical	Quality-critical systems
Use Case	User interaction	Functional requirements

Exam-Ready Short Answers

Q: What is FAST?

FAST is a requirement elicitation technique that uses facilitated workshops to gather and define system requirements through user–developer interaction.

Q: What is QFD?

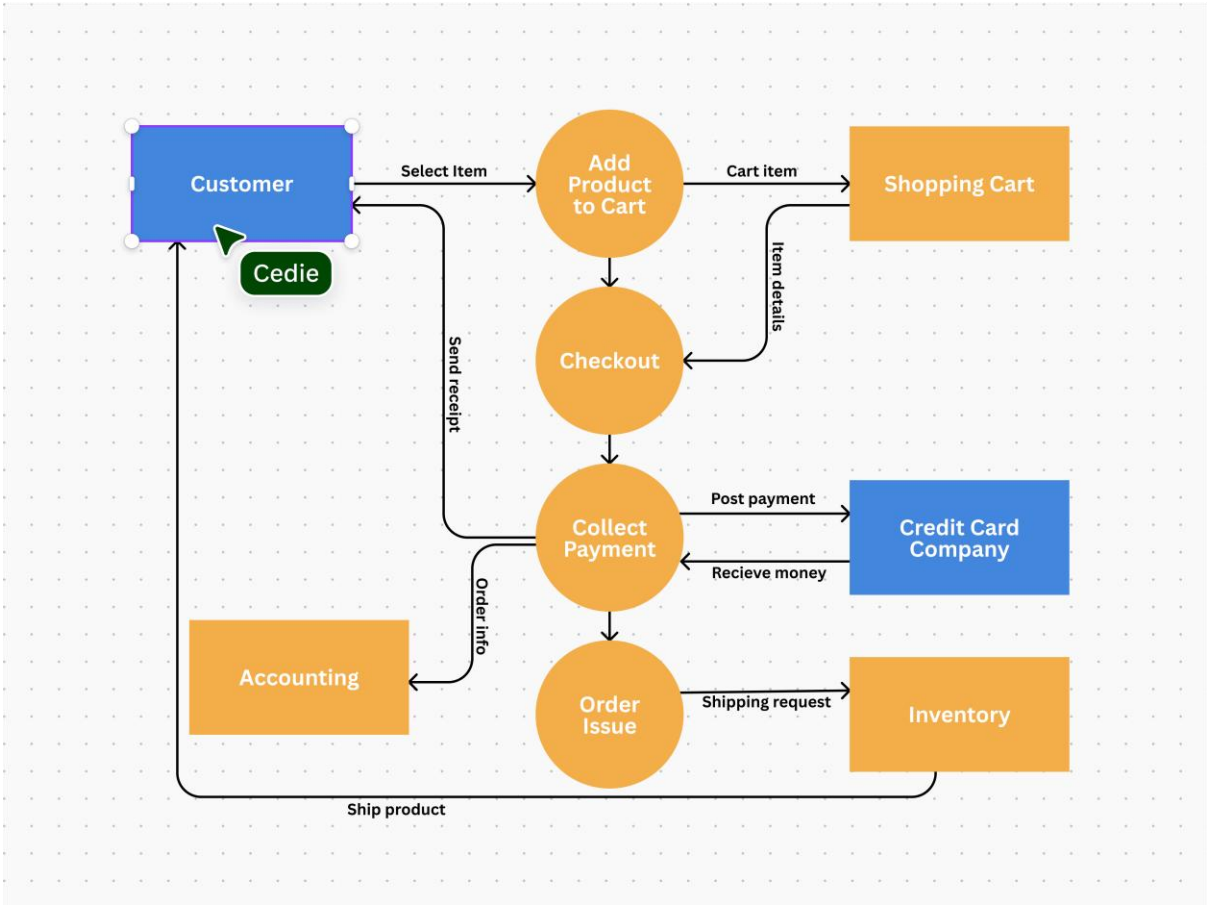
Quality Function Deployment is a technique that converts customer requirements into technical specifications using the House of Quality.

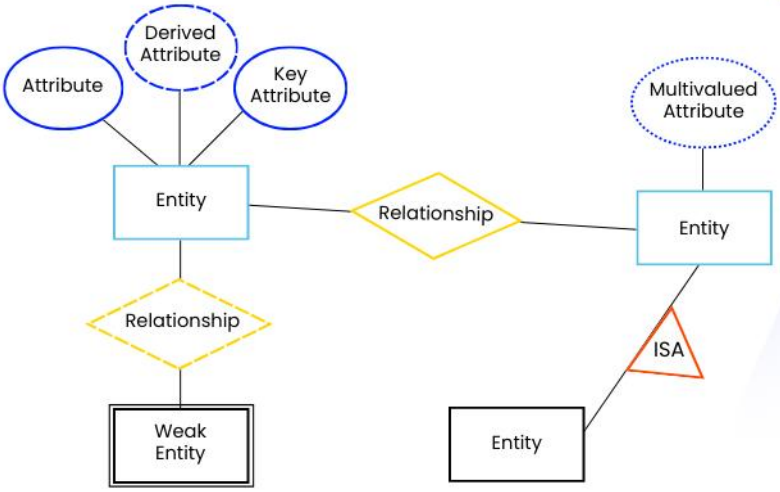
Q: What is a use case?

A use case represents a sequence of interactions between an actor and the system to achieve a specific goal.

Category 6: Requirements Analysis Tools

DFD, Data Dictionary & ER Diagrams





Why These Tools Are Used?

During **requirements analysis**, we must:

- Understand **data flow**
- Understand **data structure**
- Understand **relationships**

👉 These tools help convert **verbal requirements into clear models**.

---

1. Data Flow Diagram (DFD)

Definition

A **Data Flow Diagram (DFD)** is a **graphical representation** that shows:

How data flows through a system, how it is processed, and where it is stored.

Main Components of DFD

1. **Process** – Transforms data
2. **Data Flow** – Movement of data
3. **Data Store** – Stores data
4. **External Entity** – Source or destination of data

Types of DFD

1. Context Level DFD (Level 0)

- Shows the **entire system as one process**
- Interaction with external entities only

---

2. Level 1 DFD

- Breaks system into **sub-processes**
- Shows internal data flow

---

3. Level 2 DFD

- Further decomposition of Level 1 processes

📌 Higher levels = more detail.

Advantages of DFD

- Easy to understand
- Shows system boundaries clearly
- Helps identify missing requirements

Limitations

- Does not show control logic
- No timing information

Exam Tip

👉 Always mention **symbols + levels**.

---

2. Data Dictionary

Definition

A **Data Dictionary** is:

A centralized repository that defines **data elements, data structures, and data flows** used in the system.

Purpose

- Removes ambiguity
- Ensures consistency
- Acts as a reference

Contents of Data Dictionary

- Data name
- Description

- Data type
- Size
- Source
- Destination

Example

Customer_ID = Numeric, 10 digits

Customer_Name = Alphabetic, 30 chars

Advantages

- Improves communication
- Supports DFD accuracy
- Simplifies maintenance

Exam Tip

📌 Data Dictionary **supports DFD**.

---

3. Entity Relationship (ER) Diagram

Definition

An **ER Diagram** is a **conceptual data model** that represents:

Entities, their attributes, and relationships among them.

Main Components

1. Entity

- Object with independent existence
- Example: Student, Customer

---

2. Attribute

- Properties of entity
- Example: Name, ID, Age

---

3. Relationship

- Association between entities
- Example: Student *enrolls* Course

---

4. Cardinality

- One-to-One (1:1)
- One-to-Many (1:M)
- Many-to-Many (M:N)

Advantages

- Clear database design
- Eliminates redundancy
- Helps in normalization

Limitations

- No process logic
- Requires expertise

DFD vs ER Diagram (Very Important)

Aspect	**DFD**	**ER Diagram**
Focus	Data flow	Data structure
Shows process	Yes	No
Shows relationships	No	Yes
Used for	Functional analysis	Database design

Category 7 + 8 + 9 (Final Block)

Requirements Documentation + SRS + Requirement Management + IEEE Std for SRS

---

1. Requirements Documentation

Definition

Requirements documentation is the process of:

Systematically recording and organizing all identified and analyzed software requirements.

📌 The main outcome of requirements documentation is the **SRS document**.

Purpose of Requirements Documentation

- Acts as a **contract** between user and developer
- Provides a **reference for design & testing**
- Reduces ambiguity
- Helps in maintenance

Characteristics of Good Requirements Documentation

- Clear
- Complete
- Consistent
- Unambiguous
- Verifiable

---

2. Software Requirements Specification (SRS)

Definition

An **SRS** is:

A formal document that clearly describes **what the software system should do**, without describing **how it will do it**.

📌 Focus is on **WHAT**, not **HOW**.

---

3. Nature of SRS

The SRS should be:

1. **Correct** – Reflects actual user needs
2. **Complete** – All requirements included
3. **Unambiguous** – Only one interpretation
4. **Consistent** – No conflicts
5. **Verifiable** – Can be tested
6. **Modifiable** – Easy to update
7. **Traceable** – Each requirement can be tracked

---

4. Characteristics of a Good SRS (Very Important)

A good SRS should have:

- **Correctness**
- **Completeness**
- **Consistency**
- **Unambiguity**
- **Verifiability**
- **Modifiability**
- **Traceability**
- **Ranked for importance**

📌 These are often asked as **list / explain** questions.

---

5. Organization of SRS

Typical SRS Structure

1. **Introduction**
    - Purpose

    - Scope
    - Definitions
2. **Overall Description**
    - Product perspective
    - User characteristics
    - Constraints
3. **Specific Requirements**
    - Functional requirements
    - Non-functional requirements
4. **External Interface Requirements**
5. **Appendices**

---

6. IEEE Standard for SRS

The most commonly referenced standard is by **IEEE**.

IEEE Std 830 (Classic Reference)

(Modern revisions exist, but exams usually expect this)

Why IEEE Standard is Used

- Provides a **uniform format**
- Improves clarity
- Enhances communication
- Reduces misunderstanding

Key IEEE Guidelines for SRS

- Use clear language
- Avoid design details
- Use consistent terminology
- Make requirements testable

📌 **Important exam line:**

IEEE standard defines the **recommended structure and characteristics** of an SRS document.

---

7. Requirement Management

Definition

Requirement Management is:

The process of **handling requirement changes** throughout the software life cycle.

Why Requirement Management is Needed

- Requirements change frequently
- New user needs arise
- Business rules evolve

Key Activities

1. Requirement identification
2. Change control
3. Impact analysis
4. Requirement traceability
5. Version control

Benefits

- Prevents scope creep
- Maintains consistency
- Improves project control

Common Exam Questions

- What is SRS? Explain its characteristics
- Explain IEEE standard for SRS

- What is requirement management and why is it needed?
- Write the organization of SRS